Author: Suhas Suresha & Dewang Sultania

SECTION 1

The developer experience contract

- **Design backward:** The ideal path lets Sarah build a healthcare intake assistant by composing chat, sessions, knowledge search, guardrails, tools, evaluation, and orchestration without becoming an infra engineer.
- **Instant productivity:** `GenAIPlatform` plus `models.chat` and named prompts should yield a working assistant in minutes while the Model Service handles keys, formats, and metering.
- **Conversation memory:** Session APIs hide storage and context-window management so follow-ups like pediatric paperwork inherit prior turns automatically.
- **Organizational knowledge:** `data.search` abstracts embeddings and ranking; workflows pass queries and optional context instead of maintaining offline document pipelines.
- **Safety and tools:** Guardrails expose policy hooks; Tool Service registration lets models invoke schedulers with auth and error handling centralized.
- **Multi-step workflows:** The `@workflow` decorator expresses business sequences while the platform owns retries, state, and tracing across coordinated service calls.

SECTION 2

Code to container: one workflow, one service

- **Shared process trap:** Running every team's logic in one process couples failures, resources, and security boundaries—unacceptable for regulated or high-traffic workloads.
- **Container isolation:** Containers balance fast startup, strong boundaries, and resource limits better than raw processes or heavy VMs for many independent assistants.
- **One workflow, one deployable:** Each decorated function becomes its own service with independent release, scaling, and versioning.
- **Workflow Service operations:** The platform operator tracks images, health, autoscaling, and discovery so authors never hand-roll replica management.

SECTION 3

Exposing workflows as APIs

- **API sprawl risk:** Without standards, every team invents URLs, auth, and error shapes—frontend and partner integration becomes a coordination crisis.
- **Explicit `api_path`:** Developers choose stable public paths separate from internal workflow names so refactors do not break clients.
- **Unified domains:** Hostnames group workflows under consistent auth and response conventions while allowing organizational boundaries (clinical vs. public).
- **HTTP for externals:** Universal POST/GET semantics plus JSON (or SSE) keep browser, mobile, and partner access library-free.
- **Sync, async, stream:** Quick classification uses blocking JSON; long research uses job IDs and polling; conversational UX uses Server-Sent Events for token-by-token delivery.

SECTION 4

Communication architecture

- **Gateway as edge:** External HTTPS terminates at an API Gateway handling TLS, identity, rate limits, load balancing, and canary routing to healthy workflow replicas.
- **gRPC inside:** Service-to-service calls use compact Protocol Buffers through the same gateway as a gRPC proxy, keeping remote calls feel like local SDK methods.
- **End-to-end trace:** Maria's mobile POST flows gateway → workflow container → repeated gRPC hops to Session, Data, and Model with tracing injected at the edge.
- **Scale the planes:** Stateless gateways and per-service scaling let throughput grow without redesigning the topology from tens to thousands of requests per second.

SECTION 5

Building the SDK

- **`GenAIPlatform` entrypoint:** Resolves gateway URL from constructor, environment, or sensible default so workflow containers reach the mesh without host sprawl.
- **Lazy service clients:** Properties such as `sessions` and `models` construct gRPC clients on first use with `x-target-service` metadata for routing.
- **`@workflow` metadata:** The decorator captures name, API path, replica policy, and `response_mode` for deploy tooling that builds images and registers with the Workflow Service.
- **Consistent pattern:** Every platform service reuses one connection story, one client shape, and one deployment contract so teams learn the SDK once.